

Progression notes

Working log — last updated 2026-09-20. For the project overview, setup, and run commands, see the [README](#). I add to this page as the project grows; the final report will be a trimmed subset of it.

Current progress

Implemented

- **Bilinear reference reader:** A full-resolution sampler that every later representation is read through, so quality differences come from the representation and not the sampling code.
- **S3TC/DXT1 baseline:** Classic 4×4 block compression from scratch. It gives a fixed 6× reduction and lands around 41–43 dB, which is the bar the learned version has to beat.

For easy comparison, both implementations are abstracted to expose the same `compress(texture)` / `sample(u, v)` interface.

To implement next

- A multi-resolution feature grid plus a decoder MLP, fit per texture, then post-training 8-bit quantization.

The shared read convention

A texel's center sits at `(i + 0.5) / size`, and a coordinate `u` maps into continuous texel space as `x = u * width - 0.5`, likewise for `v / y`. Lookups outside the texture clamp to the border (replicate padding), matching PyTorch's `F.grid_sample(..., align_corners=False, padding_mode="border")`.

Bilinear sampling

Given (u, v) , find the four surrounding texels, take the fractional position (s, t) inside the cell, and blend:

$$\text{color}(u, v) = (1-s)(1-t) \cdot c_{00} + s(1-t) \cdot c_{10} + (1-s)t \cdot c_{01} + s \cdot t \cdot c_{11}$$

The four corners are named for their offsets: c_{00} is the top-left of the cell and c_{11} the bottom-right. It's vectorized over a batch of coordinates, so the same function handles a few probes or a full megapixel pass.

Correctness validated by:

- Sampling an image at its own texel centers reproduces it exactly (PSNR = ∞), which pins down the texel-center convention.
- On random coordinates it matches `F.grid_sample` (`align_corners=False`, `padding_mode="border"`) to floating-point tolerance, borders included.

S3TC / DXT1 block compression

S3TC splits the image into 4x4 blocks and spends a fixed 64 bits per block:

- Two endpoint colors, each a 16-bit RGB565 value ($2 \times 16 = 32$ bits).
- Sixteen 2-bit indices, one per texel ($16 \times 2 = 32$ bits).

In total, it takes 8 bytes per 16 texels, or 4 bits per texel. This is **6x smaller** than 24-bit RGB. A four-color palette is interpolated from the endpoints, and each texel stores the index of its nearest palette color:

$$c_2 = (2c_0 + c_1) / 3 \quad c_3 = (c_0 + 2c_1) / 3$$

When decoding, the `sample(u, v)` path decodes the image once and then bilinearly samples it with the reader above, so S3TC is read exactly like every other representation.

Picking the endpoints

The endpoints dominate block quality, so I choose them per block in two steps:

1. **Principal color axis (PCA).** Subtract the block mean, approximate the dominant direction of variation with a few power iterations on the 3×3 covariance, project the 16 texels onto that axis, and take the two extremes. This lines the palette up with the direction the block actually varies in.
2. **Least-squares refit.** Assign each texel to its nearest palette entry, then solve for the endpoints that best reconstruct those colors. Palette entries are fixed blends of the endpoints (`c2` and `c3` above), so each block is a tiny 2-variable least-squares problem with a closed form. Repeat this assign-and-refit twice.

Flat blocks, where the fit is underdetermined, just keep their previous endpoints instead of dividing by a near-zero determinant. Final endpoints are quantized to RGB565, and blocks are always written in the 4-color mode with the larger code first.

RGB565, and why the divide-by-levels shortcut isn't exact

Each endpoint channel is quantized by multiplying by its level count, rounding, and dividing back:

```
levels = (31, 63, 31)           # 5, 6, 5 bits for R, G, B
q      = round(c * levels)       # integer 5-6-5 code
c_hat  = q / levels              # back to [0, 1]
```

This is close to, but not exactly, how hardware maps a 5- or 6-bit code back to a value. Hardware expands a code by *bit replication* into 8 bits, so the true value would be `round(q * 255 / 31) / 255`. The shortcut implemented instead returns `q / 31`. Since $255/31$ isn't an integer, bit replication lands on one of 256 fixed 8-bit levels, while `q / levels` can fall between them. The difference is under a $1/255$

step and both maps are monotonic with the same half-step rounding error, so the difference is not significant.

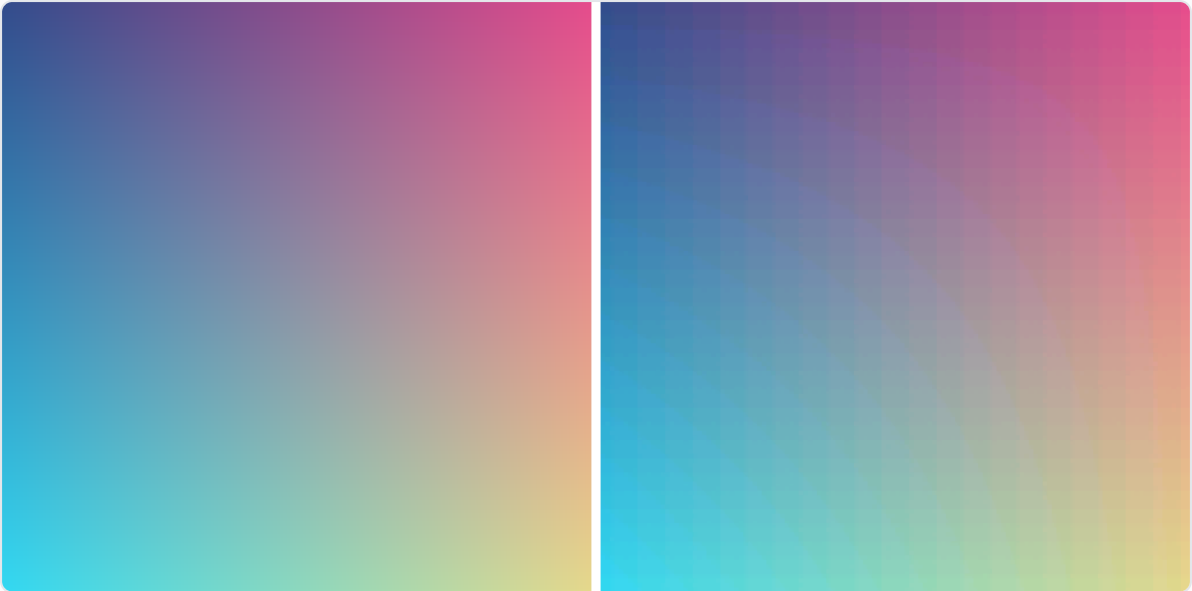
Why does green get the extra bit? The 5-6-5 split puts six bits on green, one more than red or blue. This is because green carries most of the human eye's perceived luminance (which is probably because sunlight is mostly green which is because green is between blue and red in terms of light wavelength). The Rec. 709 weights are about 0.21 R, 0.72 G, 0.07 B, so a green error is 3x as visible as the same error in red and 10x as visible as one in blue. Spending the extra level on green therefore removes the most visible banding per bit. From what I've seen, every common 16-bit RGB layout favors green because of this.

Results

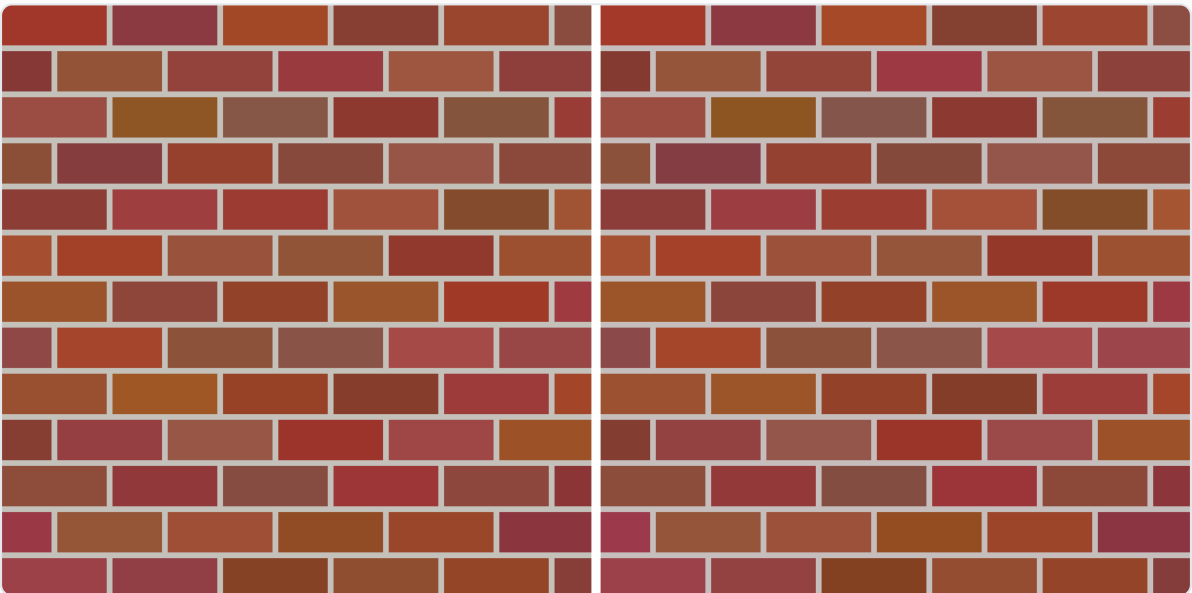
S3TC is fixed-ratio, so all three textures compress to exactly the same size. PSNR is measured on the decoded image against the original, both normalized to [0, 1] , with $PSNR = -10 \cdot \log_{10}(MSE)$.

Texture	PSNR (dB)	S3TC size	Original size	Compression
gradient (smooth, low-frequency)	42.89	128 KB	768 KB	6.00× smaller
bricks (structured, sharp edges)	41.39	128 KB	768 KB	6.00× smaller
clouds (fractal, multi-scale)	42.63	128 KB	768 KB	6.00× smaller

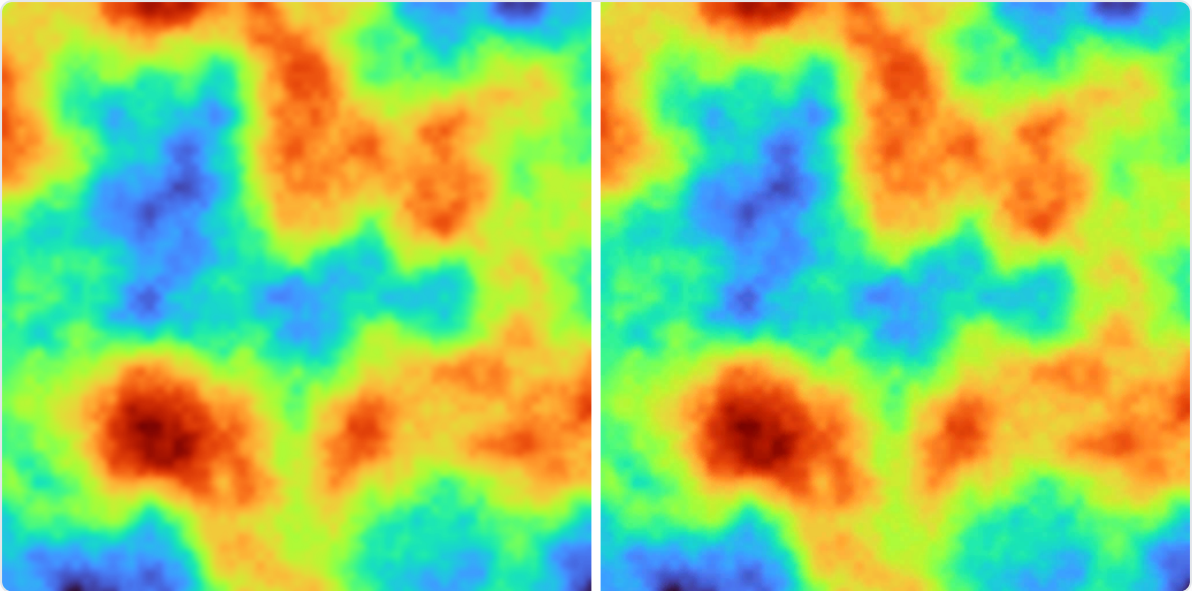
"Compression" is raw size divided by compressed size (a 6× reduction); the compressed file is one sixth of the original.



Gradient: original (left) vs. S3TC (right), 42.89 dB. The smooth ramp is the hardest case for a block format — the reconstruction shows faint block stepping.



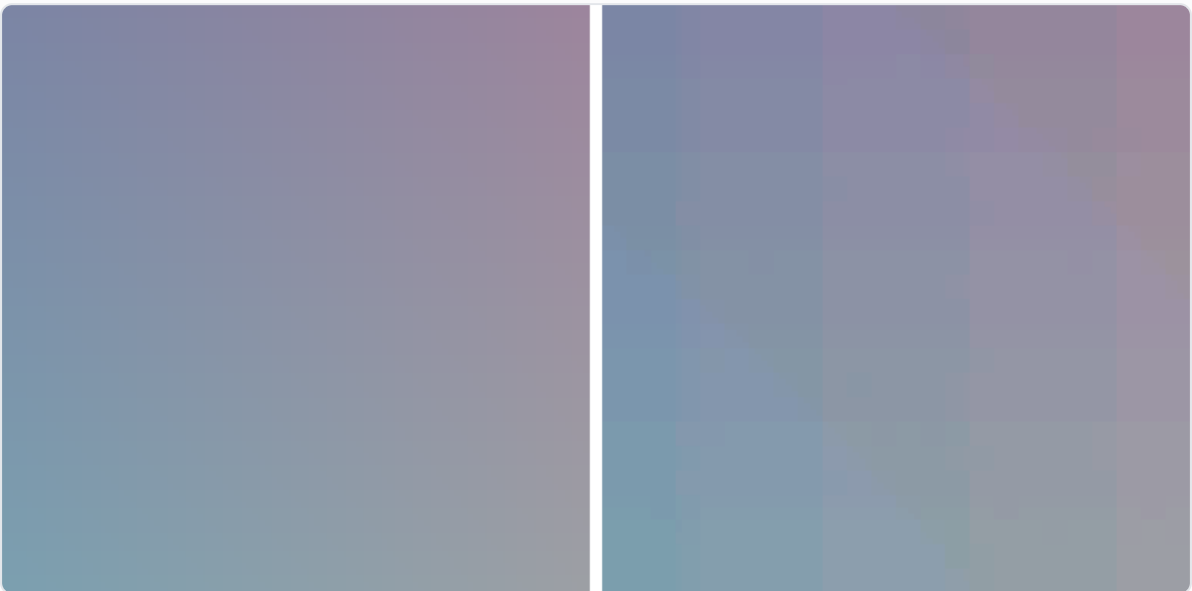
Bricks: original (left) vs. S3TC (right), 41.39 dB. Brick faces and mortar survive, but their colors shift slightly inside each block.



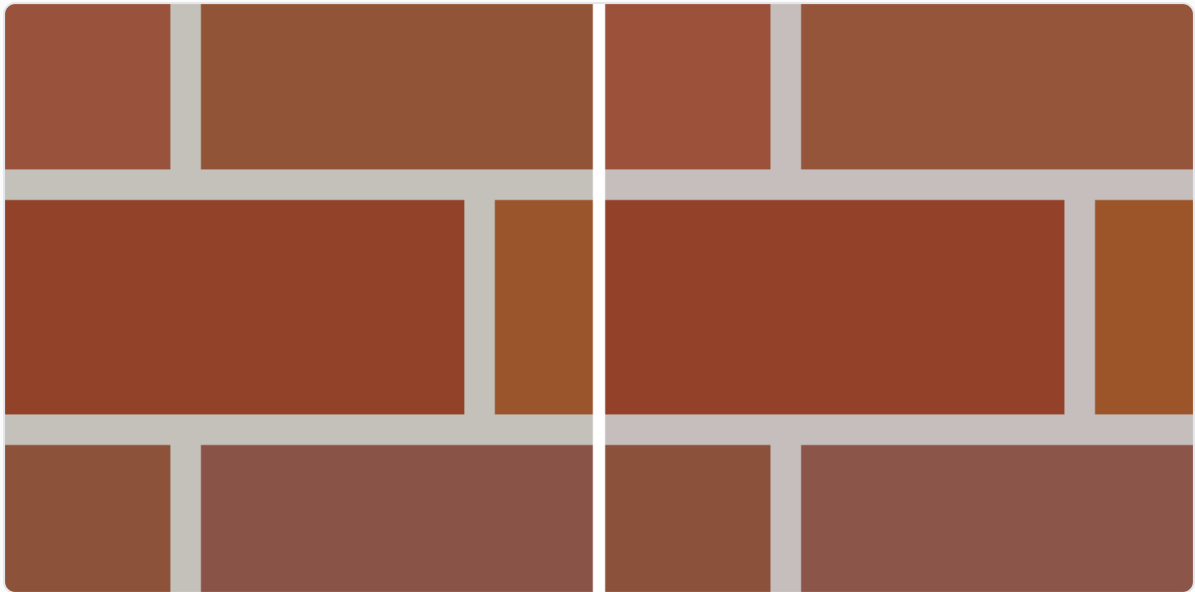
Clouds: original (left) vs. S3TC (right), 42.63 dB. The soft, low-contrast detail mostly survives, with subtle blocking.

S3TC is sometimes not good

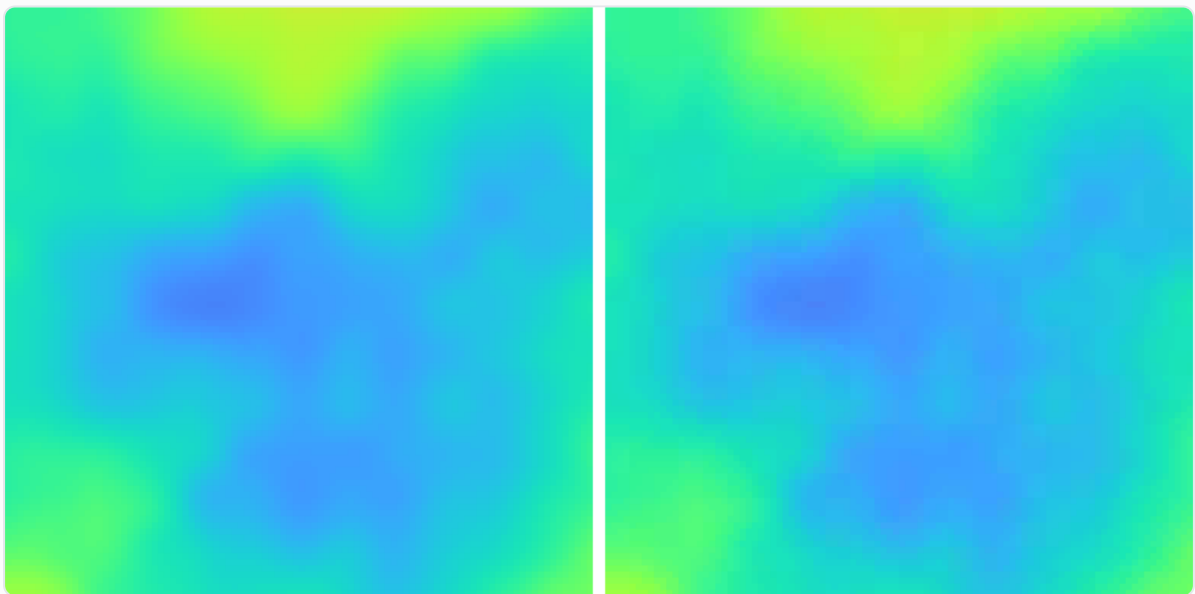
The clearest failure is smooth gradients. The crops below are 96×96 texels, nearest-neighbor upscaled 4×, so each 4×4 block becomes a 16×16 square. Figures are regenerated by `scripts/make_figures.py`.



Gradient: the original ramps smoothly; S3TC shows block-to-block banding.



Bricks: flat brick color picks up blocky shading from the palette fit.



Clouds: soft detail survives; faint blocking in low-contrast areas.

This behavior is because of the nature of S3TC and cannot really be improved by the per-block fit alone.

- The palette is four points on the segment between two endpoints. A smooth image varies in two dimensions inside a block, but the block can only represent colors near that 1-D line, so the leftover error shows up as a color step at every block boundary, causing the banding.

- High-frequency detail that doesn't align with the 4×4 grid can't be represented; edges get smeared or stepped.
- Every block costs the same 64 bits whether it's flat or busy, and nothing is shared between blocks.
- RGB565 rounds each endpoint to 5 or 6 bits before the palette is even built, adding a second error source on top of the palette fit.

At this stage the baseline is 128 KB (6×) at roughly 41–43 dB. A learned representation will replace the fixed palette with a shared feature grid plus an MLP, so its size will be tunable instead of fixed. Because of this, I will compare them by matching size then comparing PSNR and matching PSNR then comparing size.

Can the S3TC endpoint choice reduce the blocking issues?

The 4-colors-per-block limit is structural, but the baseline picks endpoints block by block with no idea what its neighbors chose, so it leaves recoverable seam on the table. I kept the bit budget and the decoder exactly the same and changed only how the endpoints are chosen, trying three neighbor-aware or global approaches in increasing cost.

Seam metric

To measure how bad/visible seams were, I made a seam error metric. For every pair of adjacent texels straddling a block boundary, compare the reconstruction's color step to the original's:

$$e = \text{mean } | (\text{rec}[a] - \text{rec}[b]) - (\text{orig}[a] - \text{orig}[b]) |$$

- Subtracting the original's own gradient means a perfect reconstruction scores 0 even on a textured image, and only added discontinuities count.
- Normalized color units, lower is better. It is not identical to perceived blocking, but it should be well-correlated.
- Baseline: gradient 0.00225, clouds 0.00690, bricks 0.00059.

- There is a floor: RGB565 rounding alone contributes about half the seam. Encoding with float endpoints drops the gradient seam from 0.00225 to 0.00119, so endpoint quantization is a real part of the artifact.

Attempt 1: Block error diffusion

- **Idea:** a block's reconstruction error on its right and bottom edges is fed forward into the target colors of the block to its right or below, before that block is fit, so errors cancel across the seam instead of stacking up.
- **Result:** gradient seam -8% and $+0.18$ dB, clouds seam -4% and $+0.14$ dB.
- **Conclusions:** the seam falls as alpha drops and bottoms out around 0.1 (mean seam $\times 0.94$ across gradient and clouds); by 0.75 it adds seam instead of removing it. PSNR moves the other way, rising with alpha until about 0.5 ($+0.28$ dB). It converges within two passes, so I set alpha 0.1 and stop early.

Attempt 2: Boundary-weighted refit

- **Idea:** refit each block's edge texels toward `neighbor_decoded + original_step`, so the block's step across the seam matches the original's, and weight edge texels higher. This targets the seam metric directly.
- **Result:** best seam on the gradient (-13%) with a small PSNR gain. On clouds it trims seam -5% but gives up 0.1 dB.
- **Conclusions:** gamma 0.5 to 0.75 minimizes the seam (mean $\times 0.91$); raising the boundary weight to 2 trims a little more but starts costing PSNR, so I leave it at 1.0. It converges after one or two passes.

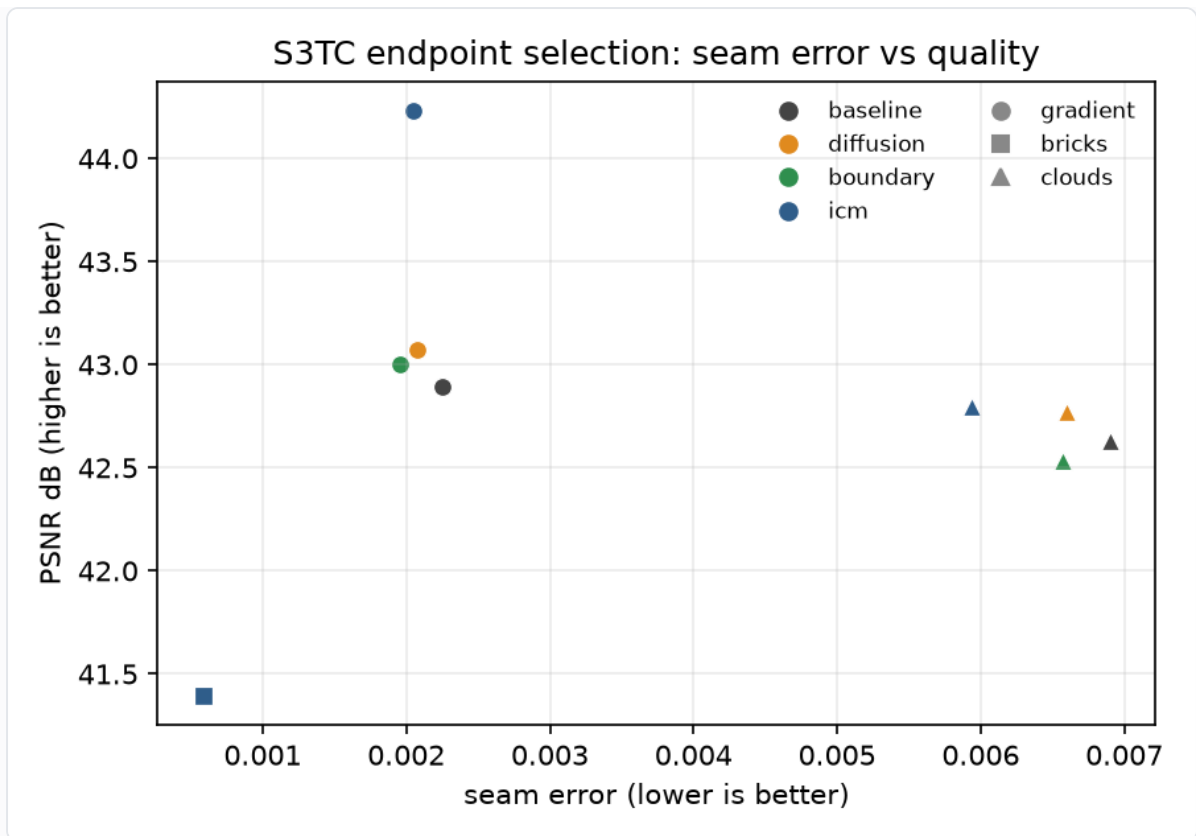
Attempt 3: Seam-energy search (ICM)

- **Idea:** iterated conditional modes. Each block tries candidate endpoint pairs (its current pair, the PCA extremes, the baseline least-squares fit, and blends toward each of its four neighbors), decodes them, and keeps the one minimizing its own squared error plus `lam × added-seam`.
- The candidates are evaluated on the *quantized* decode, so the search also finds endpoints that survive RGB565 rounding, which is why PSNR improves too.
- **Result:** best overall. Gradient $+1.34$ dB and seam -9% ; clouds $+0.16$ dB and seam -14% .

- **Conclusions:** lam sets the trade-off. Low lam (0.1 to 0.5) maximizes PSNR (+1.1 dB) while still cutting seam; lam 1 to 2 is the seam sweet spot (mean $\times 0.885$); lam 10 and above over-constrains the clouds and PSNR collapses. beta barely moves the seam but higher values favor smooth textures (up to +2.3 dB on the gradient at beta 0.8) at a small cost on clouds, so 0.5 stays balanced. Seam keeps improving out to 8 to 12 passes; 4 captures most of it.

Results

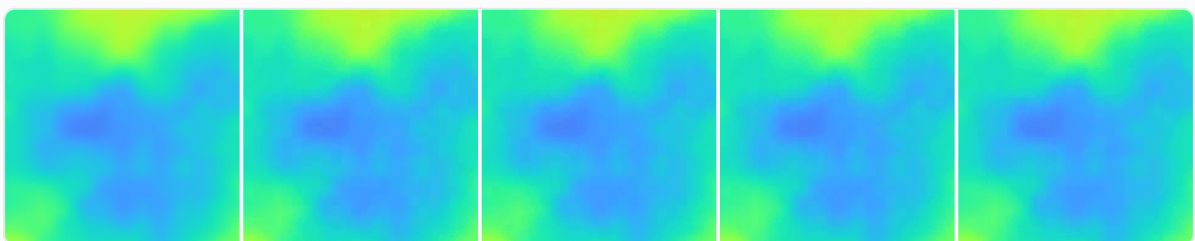
Texture	Method	PSNR (dB)	Seam error
gradient	baseline	42.89	0.002247
gradient	diffusion	43.07	0.002072
gradient	boundary	43.00	0.001959
gradient	ICM	44.23	0.002046
clouds	baseline	42.63	0.006901
clouds	diffusion	42.77	0.006595
clouds	boundary	42.53	0.006575
clouds	ICM	42.80	0.005936
bricks	all methods	41.39	0.000585



PSNR against seam error. Up and to the left is better. ICM (blue) is highest, boundary (green) is leftmost on the gradient, and the clouds points sit furthest right because that texture has the most seam to begin with.



Gradient, 4x zoom. Left to right: original, baseline, diffusion, boundary, ICM. The banding gets progressively less pronounced, most visibly in the ICM panel.



Clouds, 4x zoom, same order. ICM removes most of the visible block structure in the low-contrast areas.

Take aways

- **ICM (method 3) is the best all-rounder:** the only method that improves PSNR on every non-flat texture and cuts the seam the most on clouds, for a few passes of extra compute and no change in stored size.
- **Boundary refit (method 2) is the cheapest seam win** and wins outright on the smooth gradient, but it can trade a little PSNR for the seam on busier textures.
- **Error diffusion (method 1) is a mild, safe nudge** only at low strength.
- **Bricks is unchanged by all three.** Its blocks are flat, so the baseline already has essentially no added step to fix. A useful sanity check that the methods only touch recoverable seam.
- **Quantization is the remaining floor.** None of these remove the roughly half of the seam that comes from RGB565 rounding. Coordinating the endpoint quantization itself, or searching the RGB565 lattice globally, is probably the best next move to reduce blocking.
- All of this is free in stored size: the representation is still exactly 8 bytes per block (a fixed 6×), only the endpoint choice changed.

Log

2026-09-20 Project scaffold, environment, and CUDA setup. Bilinear sampler with the shared read convention, verified against `grid_sample`. S3TC/DXT1 baseline with PCA endpoints and least-squares refits; 6× fixed at ~41–43 dB. Figures and this page added.

2026-09-20 Added a cross-boundary seam metric and three seam-aware endpoint-selection variants (block error diffusion, boundary-weighted refit, seam-energy ICM). ICM improves PSNR and cuts the seam the most; boundary refit gives the best seam on smooth gradients; error diffusion is a mild nudge. All keep the fixed 6× size.
